

Double Gumbel Q-Learning

백승언, 김윤하, 박민제

14 Jul, 2024

- **Introduction**
 - Backgrounds
 - Limitation in mean-squared Bellman error
- **Double Gumbel Q-Learning(DoubleGum)**
 - Related works
 - DoubleGum
- **Experiment results**

Introduction

Backgrounds (I) – Central Limit Theorem vs Extreme Value Theorem

● Central Limit Theorem (CLT)

- **Theorem:** Let $X_1, \dots, X_n$ be i.i.d. random variables with mean μ and variance σ^2 . Then,

$$\lim_{n \rightarrow \infty} \sqrt{n}(\bar{X}_n - \mu) \sim \mathcal{N}(0, \sigma^2)$$

- **Justification of Gaussian error in regression via CLT**

- In general, regression model(e.g., $y = f_\theta(x) + \epsilon_\theta$) assumes that the error term follows a homoscedastic normal distribution

$$\epsilon_\theta \sim \mathcal{N}(0, \sigma^2)$$

- If ϵ_θ is the sum of various independent noises, even if each single noise does not follow a normal distribution, the sum, ϵ_θ , could be approximated as a normal distribution by CLT

● Extreme Value Theorem(EVT)

- Let $X_1, \dots, X_n$ be i.i.d. random variables with cdf F and let $M_n = \max\{X_1, \dots, X_n\}$. denote the sample maximum.
 - In theory, the exact dist. of the maximum could be derived as

$$P\{M_n \leq z\} = P\{X_1 \leq z\} \times \dots \times P\{X_n \leq z\} = (F(z))^n$$

Backgrounds (II) – Gumbel distribution

● Extreme Value Theorem(EVT)

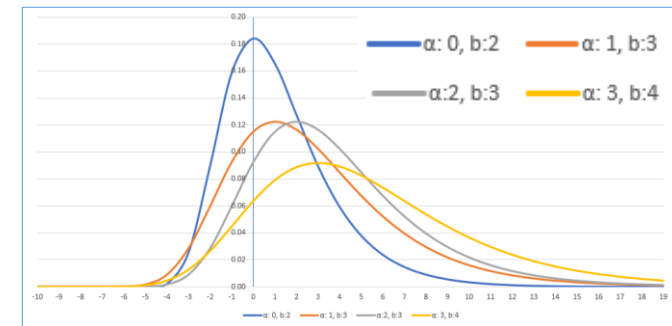
- Let $X_1, \dots, X_n$ be i.i.d. random variables with cdf F and let $M_n = \max\{X_1, \dots, X_n\}$. denote the sample maximum.
 - In theory, the exact dist. of the maximum could be derived as $P\{M_n \leq z\} = P\{X_1 \leq z\} \times \dots \times P\{X_n \leq z\} = (F(z))^n$
- In practice, Fisher-Trippett-Gnedenko theorem provides an asymptotic results

$$P\left\{\frac{M_n - b_n}{a_n} \leq z\right\} \rightarrow G(z), \quad \text{with } a_n > 0 \text{ and } b_n \in \mathbb{R},$$

as $n \rightarrow \infty$, $G(z) \propto \exp\left[-(1 + \gamma \cdot z)^{-\frac{1}{\gamma}}\right]$, where the γ depends on how steeply of the tail diminish

● Gumbel distribution $\mathcal{G}(\alpha, \beta)$

- Type I generalized extreme value distribution, $G(z)$ for $\gamma = 0$
 - α : location parameter, β : scale parameter.
 - PDF $p(x) = \frac{1}{\beta} e^{-(z+e^{-z})}$, $z = \frac{(x-\alpha)}{\beta}$.
 - $\mathbb{E}[X] = \alpha + \beta\gamma$ (γ : Euler-Mascheroni constant)
 - $\max_x p(x) = \alpha$
 - CDF $P(x) = \frac{1}{\beta} e^{-e^{-z}}$
 - $\text{Var}[X] = \frac{\pi^2}{6} \beta^2$
 - $X \sim \mathcal{G}(\alpha_X, \beta), Y \sim \mathcal{G}(\alpha_Y, \beta) \rightarrow X - Y \sim \text{Logistic}(\alpha_X - \alpha_Y, \beta)$



Gumbel dist. according to params

Limitation in mean-squared Bellman error (I)

- **The object of Reinforcement Learning(RL) is to learn optimal policy π^* , which maps the maximally rewarding decision a^* corresponding state s**
 - For this reason, Q-function $Q(s, a)$, which estimates cumulative reward for every pair (s, a) was proposed by Watkins(1992)
 - $Q_\pi(s, a) = \mathbb{E}_{\pi, P}[\sum_{j=i}^n \gamma^j r_j \mid s_i = s, a_i = a]$, denoting $r(s_j, a_j, s_{j+1})$ as r_j , where the discount factor $\gamma \in]0, 1[$
 - In every Markov Decision Process(MDP), an optimal policy π^* makes maximally rewarding decision following $s \rightarrow \operatorname{argmax}_a Q^*(s, a)$
- **Optimal Q-function, $Q^*(s, a)$ is specified as the unique fixed point of the Bellman Optimality Equation of the MDP(Bellman, 1957)**
 - $Q^*(s, a) = \mathbb{E}_{p(s'|s, a)}[r + \gamma \max_{a'} Q^*(s', a')]$, over all s and a .
- **In Deep Reinforcement Learning(DRL), Q^* is approximated by a neural network Q_θ with params θ .**
 - The parameter θ is updated by a sequence of optimization problems whose i^{th} state iteration is given by
 - $\theta_{i+1} = \operatorname{argmin}_\theta \mathcal{L}(Q_\theta, y_{\bar{\theta}})$, where $y_i(s, a) = \mathbb{E}_{p(s'|s, a)}[r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a')]$
 - Typically, the loss function $\mathcal{L}(Q_\theta, y_{\bar{\theta}}) = \mathbb{E}_{s, a} (Q_\theta(s, a) - y_{\bar{\theta}}(s, a))^2$ is the **Mean-Squared Bellman Error(MSBE)**.

Limitation in mean-squared Bellman error (II)

- **The authors derived the MSBE with two assumptions**

- $\epsilon_{\theta,s,a} \sim^{i.i.d} \mathcal{N}(0, \sigma^2)$, where $\mathcal{N}$ is a homoscedastic normal due to CLT
 - $\epsilon_{\theta,s,a} := Q^*(s, a) - Q_{\theta}(s, a)$
- $Q^*(s, a) := \mathbb{E}_{p(s'|s, a)} \left[r + \gamma \max_{a'} Q^*(s', a') \right] \approx y_{\bar{\theta}}(s, a)$, where θ are the params at an arbitrary state
- These assumptions uncovered the implicit assumption that TD error follows a homoscedastic normal
 - $\epsilon_{\theta,s,a} := y_{\bar{\theta}}(s, a) - Q_{\theta}(s, a) \sim^{i.i.d} \mathcal{N}(0, \sigma^2)$

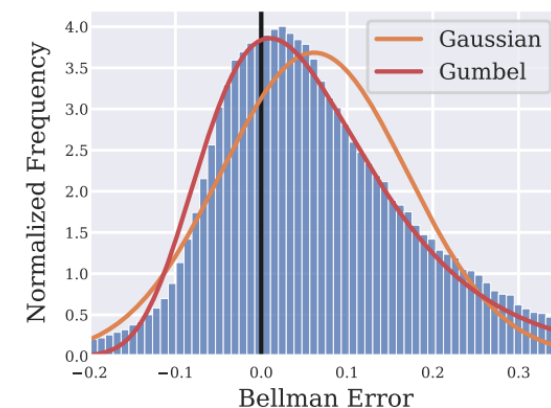
- **They insisted that above two assumptions used to derive the MSBE are theoretically weak**

- First, the CLT states that i.i.d samples of an underlying distribution tends towards a homoscedastic normal but do not account for the form of the underlying distribution
 - When the max over a finite number of i.i.d samples with unbounded support is taken, the EVT gives the limiting distribution of the resultant random variable as a Gumbel distribution, defined here as $g_{\theta,s'} \sim \mathcal{G}(\alpha, \beta)$
- Secondly, there is no analytic justification for $Q^*(s, a) \approx y_{\bar{\theta}}(s, a)$.

Double Gumbel Q-Learning (DoubleGum)

● Extreme Q-Learning: Deep Q-Learning with one homoscedastic Gumbel noise

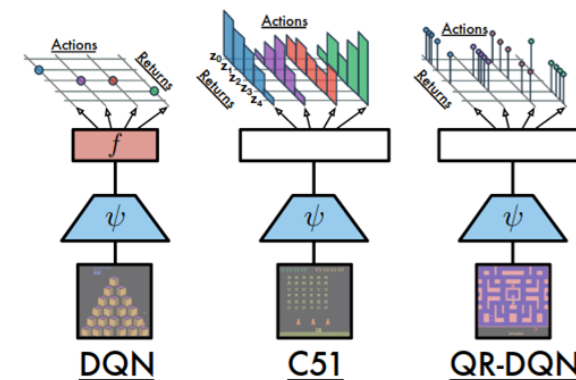
- The authors derived extreme Q-learning(XQL) from the Soft Bellman Equation backups
 - $Q(s, a) \leftarrow \mathbb{E}_{p(s'|s, a)}[r(s, a, s') + \gamma V^{soft}(s)], \text{ where } V^{soft}(s) = \beta \log \sum_{a'} \mu(a'|s') \exp\left(\frac{Q(s', a')}{\beta}\right)$
- They analyzed the Bellman backup, and showed the ϵ_θ follows the Gumbel distribution
 - Then, they presented a method of estimating value using Gumbel regression
 - The soft optimal value V^* can be directly estimated using Gumbel regression by setting the fixed β



Empirical Bellman errors

● Distributional RL

- In distributional RL, the cumulative reward of a chosen action at state is modeled with the full distribution rather than expectation of it, $Z_\theta(s, a) := \frac{1}{N \sum_{i=1}^N} \delta_{\theta_i}(s, a)$
 - So that the model can capture its intrinsic randomness instead of just first-order moment (high-order moments, multi-modality in state-action value function)
- Existing distributional RL algorithms
 - C51: Pre-determination of the supports of return distribution and then training the categorical distribution.
 - QR-DQN: To avoid pre-determined supports, and decrease the theory-practice gap, introducing the quantile regression



Distributional RL algorithms

DoubleGum (I)

- First, the authors assumed that the noise(TD error) is a heteroscedastic Gumbel noise with state-dependent spread
 - $Q_\theta(s, a) = Q^*(s, a) - g_{\theta,a}(s), \quad g_{\theta,a}(\cdot) \sim^{i.i.d} \mathcal{G}(0, \beta_\theta(\cdot)), \quad \text{for all } a \in \mathcal{A}$
- Next, they introduced the soft value function V_θ^{soft} as follows,
 - $\max_a Q^*(s, a) = \max_a [Q_\theta(s, a) + g_{\theta,a}(s)] = \beta_\theta(s) \log \int \exp\left(\frac{Q_\theta(s, a)}{\beta_\theta(s)}\right) da + g_\theta(s)$
 $= V_\theta^{soft}(s) + g_\theta(s), \quad \text{where } g_{\theta,a}(\cdot), g_\theta(\cdot) \sim^{i.i.d} \mathcal{G}(0, \beta_\theta(\cdot))$
- Then they yielded a new noise model of the Soft Bellman Optimality Equation with two function approximators and two heteroscedastic Gumbel noise sources
 - $Q_\theta(s, a) + \mathbf{g_{\theta,a}(s)} = \mathbb{E}_{p(s'|s, a)}[r + \gamma V_{\bar{\theta}}^{soft}(s') + \gamma \mathbf{g_{\bar{\theta}}(s')}]$
- Finally, they developed the Soft Bellman Equation with heteroscedastic logistic noise since the difference of two Gumbel random variables with follows logistic dist.
 - $Q_\theta^{new}(s, a) + l_{\theta,a}(s) = \mathbb{E}_{p(s'|s, a)}[r + \gamma V_{\bar{\theta}}^{soft,new}(s')], \quad l_{(\theta,a)}(\cdot) \sim \mathcal{L}(0, \beta_\theta(\cdot))$
 - $Q_\theta^{new}(s, a) = Q_\theta(s, a) + g_\theta(s), \quad V_\theta^{soft,new}(s) = V_\theta^{soft}(s) + g_\theta(s)$

DoubleGum (II)

- To learn θ , they introduced moment matching by MLE of a heteroscedastic normal due to lacks a sufficient statistical estimator of heteroscedastic logistic dist.

- They assumed that $y_{\bar{\theta}}^{soft}(s, a) \sim \mathcal{N}(Q_{\theta}^{new}(s, a), \sigma_{\theta}(s))$ and learn θ by

- $\min_{\theta} \left[\log \sigma_{\theta}(s) + \left(\frac{y_{\bar{\theta}}^{soft}(s, a) - Q_{\theta}^{new}(s, a)}{\sigma_{\theta}(s)} \right)^2 \right]$, and recovering the logistic location and spread as $\beta_{\theta}(s) = \frac{\sigma_{\theta}(s)\sqrt{3}}{\pi}$

- Computing the bootstrapped targets

- Discrete control

- In discrete control, the resultant bootstrapped target is $y_{\theta}^{discrete}(s, a, r, s') = r + \gamma V_{\theta}^{soft, new}(s')$

- Continuous control

- In continuous control, the bootstrapped target is $y_{\theta}^{continuous}(s, a, r, s') = r + \gamma \mathbb{E}_{\pi_{\phi}(a'|s)}[Q_{\theta}^{new}(s', a')] + \gamma c \sigma_{\theta}(s')$
 - They dealt with c as a hyperparameter named the pessimism factor.

● Network architecture

- They learnt σ with variance networks
 - In discrete control, $\sigma_\theta(s)$ is a variance head added to the Q -network.
 - In continuous control, $\sigma_\theta(s, a)$ is approximated
- They utilized the target network in both the discrete control and the continuous control

● Pseudo code

- Initialization of the replay buffer, neural network
 - $\mathcal{D} = \emptyset, \theta_0, \phi_0$
- Action selection with policy
 - $\pi(s) = \max_a Q^{\text{new}}(s, a)$ in discrete control
 - $\pi(s) = \pi_\phi(s)$ in continuous control
- Update value and policy functions
 - Update the target network with EMA

Algorithm 1: DoubleGum: An Off-Policy Deep Q -Learning Algorithm

Input: MDP $(\mathcal{S}, \mathcal{A}, r, p)$, replay buffer $\mathcal{D} = \emptyset$ (Lin, 1992), initial parameters θ_0 (and ϕ_0 in continuous control), initial state s , learning rates η_θ and η_ϕ , target network EMA rate η_ψ , batch size n .

Output: Parameters θ (and ϕ in continuous control)

```
1 if discrete control then
2   define  $\pi(s) = \max_a Q_{\theta_i}^{\text{new}}(s, a)$  at iteration  $i$ 
3   define  $y = y^{\text{discrete}}$ 
4   define  $\sigma = \sigma_{\theta_i}(s)$  at iteration  $i$ 
5 else if continuous control then
6   define  $\pi(s) = \pi_{\phi_i}(s)$  at iteration  $i$ 
7   define  $y = y^{\text{continuous}}$ 
8   define  $\sigma = \sigma_{\theta_i}(s, a)$  at iteration  $i$ 
9  $\psi_0 \leftarrow \theta_0$ 
10 for iteration  $i$  do
11    $a \leftarrow \pi(s)$ 
12    $s' \sim p(s' | s, a)$ 
13    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s, a, r(s, a, s'), s')\}$ 
14    $s \leftarrow s'$ 
15    $\theta_{i+1} \leftarrow \theta_i - \eta_\theta \nabla_{\theta} \mathbb{E}_{(s,a,r,s')_{1:n} \sim \mathcal{D}} [J_{\theta, y, \psi_i, \sigma}(s, a, r, s')] \Big|_{\theta_i}$ 
16    $\psi_{i+1} \leftarrow \eta_\psi \psi_i + (1 - \eta_\psi) \theta_{i+1}$ 
17   if continuous control then
18      $\phi_{i+1} \leftarrow \phi_i + \eta_\phi \nabla_{\phi} \mathbb{E}_{(s,a,r,s')_{1:n} \sim \mathcal{D}} [Q_{\theta_i}^{\text{new}}(s, \pi_{\phi}(s))] \Big|_{\phi_i}$ 
```

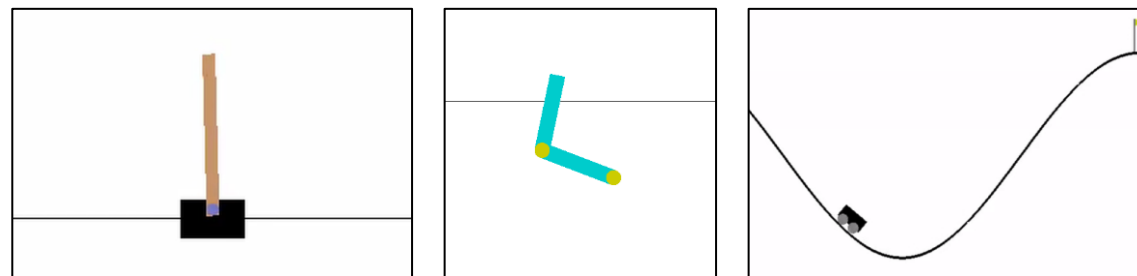
Pseudo code of DoubleGum

Experimental Results

Experimental setting

● Environments

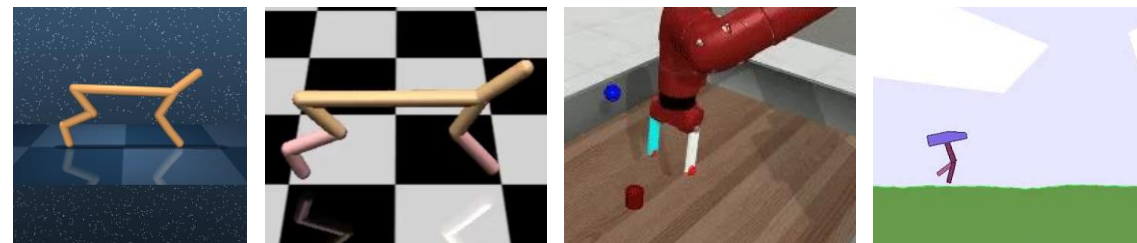
- Discrete control
 - CartPole
 - Acrobot
 - MountainCar
- Continuous control
 - DeepMindControl: 11 tasks
 - MuJoCo: 5 tasks
 - MetaWorld: 15 tasks
 - Box2D: 2 tasks



Discrete control benchmarks

● Model settings

- Default pessimism factor is $c = -0.1$
 - The authors also tuned the pessimism factor per-suite



Continuous control benchmarks

● Evaluation metrics

- InterQuantile Mean (IQN) over 12 runs, each one with a different randomly initialized seed

Experiment results (I) – Discrete control

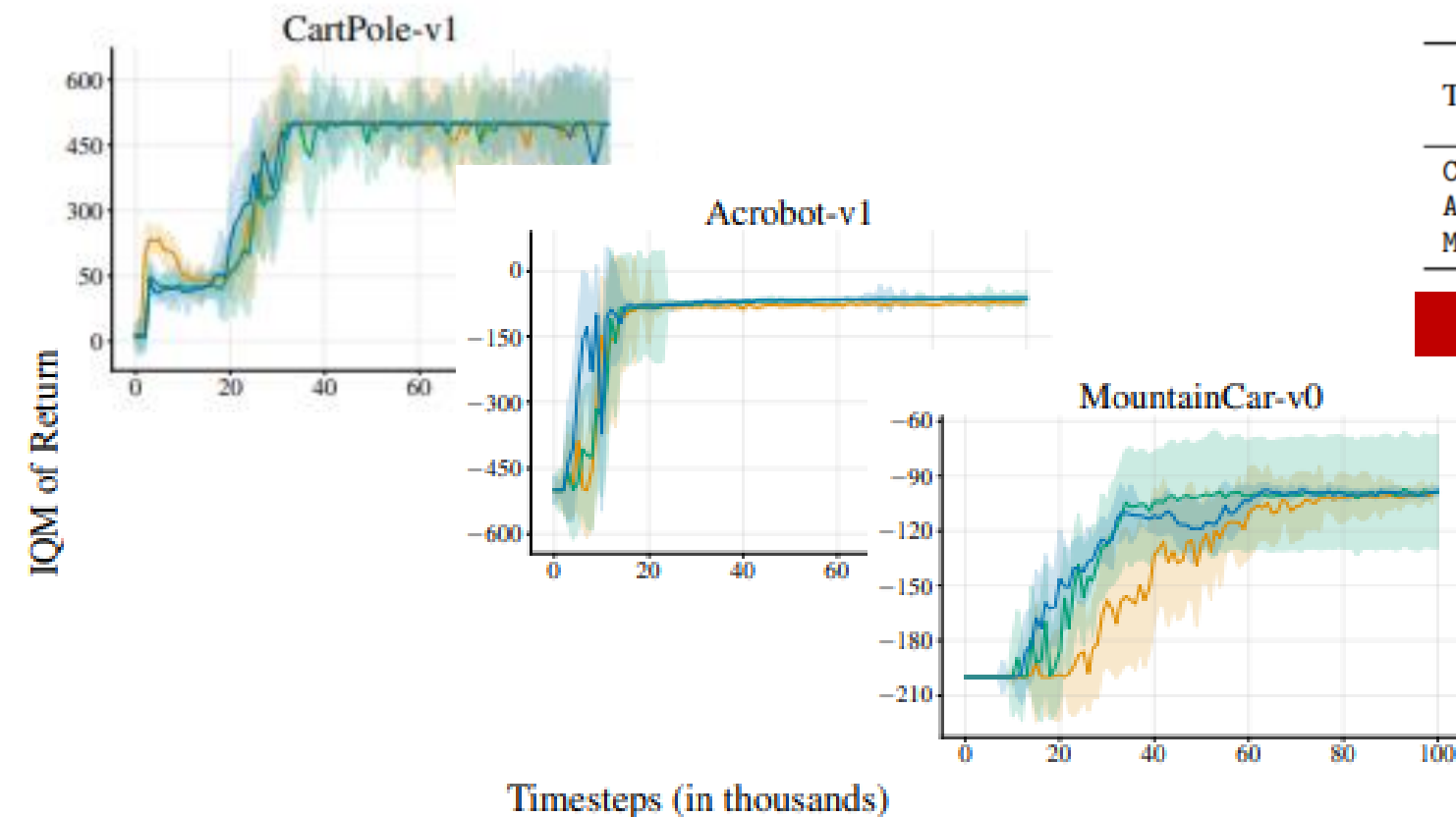
● Comparison with the previous methods

- DoubleGum obtained better sample efficiency than baselines, but not significant performance

-  DoubleGum

-  DQN

-  Dueling DDQN



Task	Score at 100K timesteps (IQM over 12 seeds)		
	DoubleGum (ours)	DQN	DuelingDDQN
CartPole-v1	500 ± 113.4	475 ± 105.5	496.9 ± 89.1
Acrobot-v1	-62.78 ± 1.775	-73.52 ± 5.191	-64.12 ± 17.15
MountainCar-v0	-98.17 ± 2.45	-99.37 ± 5.914	-98.75 ± 30.73

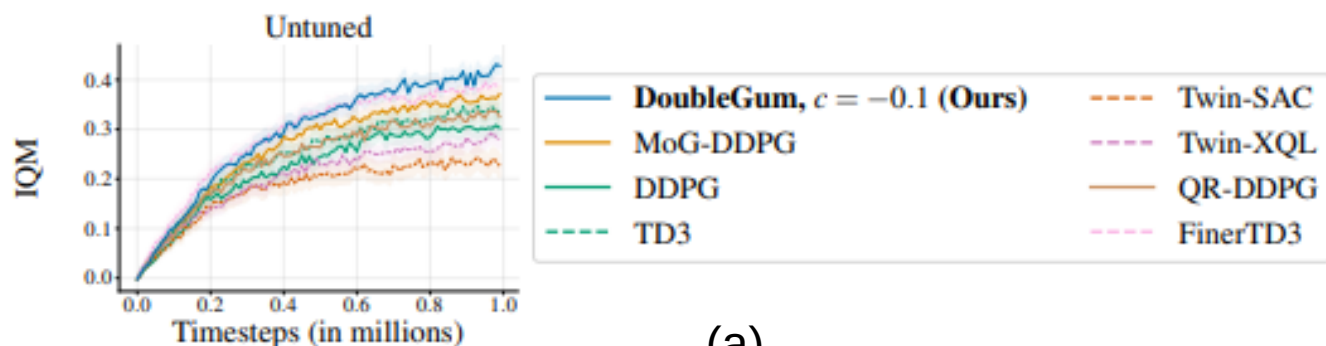
Numerical Results at 100k timesteps

IQM of returns $\pm$ standard deviation over 12 seeds

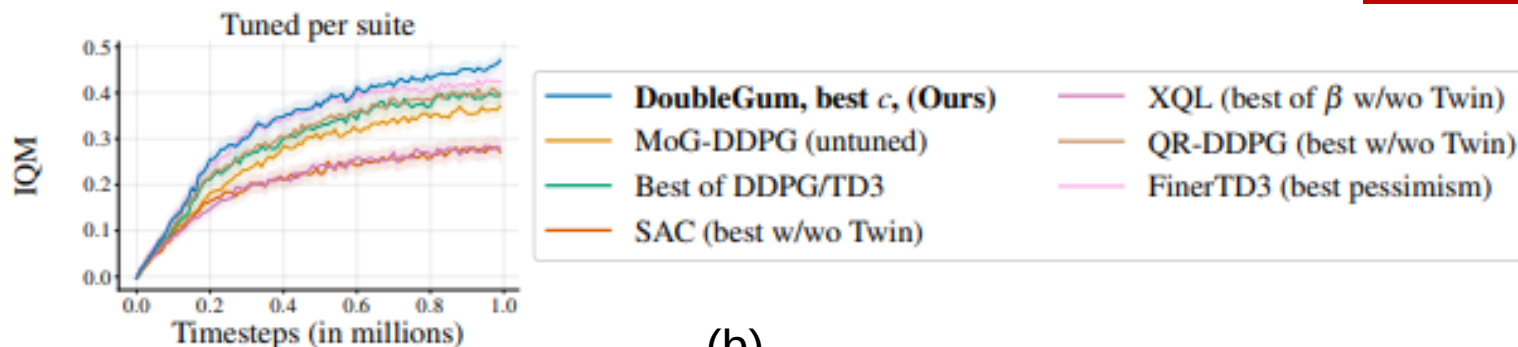
Experiment results (II) – Continuous control

● Comparison with the previous methods

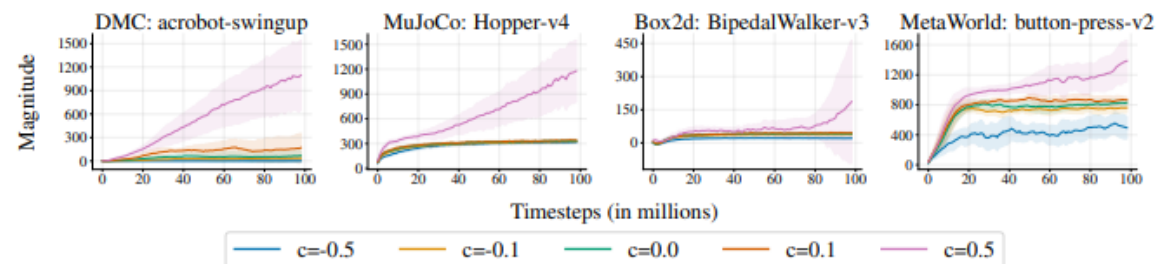
- DoubleGum outperformed all baselines in aggregate over 33 tasks, both (a) using default pessimism settings and (b) using task-specific adjusted pessimism settings



(a)



(b)



The effect of pessimism factor on the target-Q in cont. control

IQM of returns with (a) default pessimism factor and (b) tuned per-suite factor

Thank you!

Q&A